The HTTP protocol

History

The first web server

- Uses the first version of HyperText Transfer Protocol (HTTP)
- To transfer content descibed in HyperText Markup Language (HTML)
- Tim Berners-Lee and his team (CERN)
- Around 1990



Figure 1: The first web server at CERN

The first web-server is created around 1990 by a CERN worker Tim Berners-Lee. The main purpose of the web was to enable the sharing of achievements between research groups. The web consists of many protocols including HTTP (HyperText Transfer Protocol) which is a network protocol to transport HTML (HyperText Markup Language) documents across the internet. HyperText is a text which contains HyperLinks or simply links (references to other data). This is the concept behind the structure of the World Wide Web, where the media used to describe the HyperText is usually HTML. To access the WWW a client called web browser is needed. A web browser supports at least three standards: URL, which is a string representation of the location of a resource, HTTP which is the most important network protocol used to transfer a resource, and HTML, to present the content for the user, and make it possible to follow the links in the document.

Further versions

- 1991: HTTP v0.9
  - First documented version

  - Only one request method (GET)
  - The response is a pure HTML document
- 1996: HTTP v1.0
  - Request and response headers
  - Described in RFC1945
  - The first widespread version
- 1997: HTTP/1.1
  - Originally specified by RFC2068
  - Updated in 1999, current version defined in RFC2616
  - Support for named virtual servers
  - Uses permanent connections by default

The first documentation described the first implementation of the protocol made by Tim Berners-Lee and his team. Compared to recent versions, it was a very simple protocol. It was barely enough to request a document from a server and send back its content as the response to the client. In 1996, HTTP was standardized by the IETF (Internet Engineering Task Force) as RFC1945. RFC stands for Request for Comments, these documents are usually describing protocols and concepts of the Internet and computer networks in general. This version had the ability to transfer meta-information in the form of request and response headers in either direction, and to upload data, such as a filled-in form or file in the body of the request message. Version 1.1 is an extended and backward-compatible variant of the previous. An important change is to support the so-called named virtual hosts, by making the Host-header mandatory in the request messages. With this technique it is possible to serve multiple websites from a single network address. Another significant change is that it uses permanent connections by default. This method helps to reduce the overall time of requests to a single server by leaving the communication channel open between them.

REST

- Representational State Transfer
- In 2000 by Roy Fielding
- Client-server architecture
- WWW implements REST

The client-server model defined by the WWW is theorized by Roy Fielding in his PhD theses in 2000. The architectural style of WWW can be reused in other applications, most importantly in web services (called RESTful WebServices).

Architecture

Protocol

- Application layer protocol
- Over TCP/IP, default server port: 80
- Text based
- Stateless

HTTP is an application layer protocol, like FTP or SSH. It means that other lower level protocols are needed to transmit HTTP messages in a computer network. Although it is possible with many of protocols, the web traditionally uses its existing TCP/IP infrastructure. The default port of a web server is 80, however it is not necessary, but affects the address of the served website. As the message headers, and many times the message body is also encoded in plain text, the protocol is relatively easily debuggable. The protocol itself is stateless, so the sever does not have to store any data between receiving multiple requests from the same client. Nevertheless, as we will see later, many applications need the existence of some server-side state.

REST Architecture

- Client-server
- Stateless
- Cacheable
- Layered
- Code-On-Demand
- Uniform Interface

There is a dedicated client and server side. The client uses a uniform interface to get a resource. The connection is stateless, there is no context stored on the server between requests. The resources can be defined to be cacheable or not. The connection is layered, a client cannot tell whether is connected to the server which stores the resource or it connects to a intermediate server (proxy). The functionality of the client can be extended by client-side code (such as JavaScript).

Uniform interface means that the there are separate interfaces to represent how the resource can be accessed (URI in case of WWW), how a resource is represented (HTML in case of WWW) and transferred (HTTP).

Transaction

- Client-server communication
- Based on request-response pairs (transactions)

  - A client initiates a request
  - The server responds to it
  - Pull protocol
  - Push is also possible with some techniques

An HTTP transaction is a single two-way communication between a client and a server. The communication is always initiated by the client, in the form of a request. This property makes it a so-called pull protocol. The server must send exactly one response to each request. By using special techniques, for example streaming or polling, server push can also be simulated. In this case it seems that the server notifies the client about some server-side event. Streaming means the server basically tries to keep the connection open and it keeps sending the data. In the polling case the client queries the server repeatedly about changes or events at some intervals.

Session

- Multiple transactions can be chained to form a conversation
- This is usually called as a session
- Server may store temporary data on behalf of the client
  - Application-dependent state state (e.g. shopping cart)
  - The client is needed to be identified uniquely

A typical web application needs more than one transaction to do its work. The main reason is the interactivity of web: between the requests the user may or should enter some data, etc. This way the sequence of transactions will build up an entire conversation, known as a session. A later transaction is often uses data from a previous, so maintaining a state for every client may be needed on the server side. As the protocol itself is stateless, the client must identify itself somehow to access the context saved in an earlier transaction. It is possible e.g. using cookies as we will see later.

Locating resources

- To request a resource, its location must be known
- A location is expressed with a Uniform Resource Locator (URL)
  - e.g. `http://inf.elte.hu/`
- It is the special case of a Uniform Resource Identifier (URI)
  - Which can also be a Uniform Resource Name (URN)
  - A URI which identifies a resource
  - e.g. `urn:isbn:0-486-27557-4`
  - Does not describe how to obtain a representation of the resource

URI (Uniform Resource Identifier) is a protocol to identify a resource. A URI can be also URL or URN. It is a URL if it tells how a representation of the resource can be obtained. URN is an identifier which is a unique name of a resource. Such a unique name is ISBN for books. An ISBN number identifies a book without guidance how it can be obtained. However an WWW address is a URI which is a URL because it tells the exact way to access the resource, however it might not be the only way to access it, so it is not a unique identifier.

Anatomy of URI

- Relative, e.g. `../index.html`
- Absolute, e.g. `http://inf.elte.hu/`
- Generally described in RFC3986
- Depends on the scheme

```
url:                scheme:hierarchical_part[?query][#fragment]
hierarchical_part:  [user[:password]@]host[:port][/path/...]
query:              name=value[&name=query...]
```

For instance: http://john@localhost:8080/foo?id=123#chapter2

Although the syntax of URI's can be generally described, in most of the cases not all of its parts are used. The actual application is marked in the scheme part as a protocol identifier. For example, in the case of HTTP, the scheme `http://` used, and the user name and password parts are omitted. However, in case of e.g. the file transfer protocol (FTP) the scheme is `ftp://` and the fragment and query part is usually meaningless.

Message structure

Request

- Requests are sent from clients to servers
- Consists of three parts:
  1) Request line
  2) Request headers
  3) Optional message body after an empty separator line
- Line ending is CR LF (\r\n)

```
GET /index.html HTTP/1.1
Host: www.example.com
Accept: text/html
```

```
User-Agent: Mozilla/5.0 (Windows; en;) Firefox/3.6.13

[Optional request body]
```

The request line contains the request method, the URL and protocol version. In this case, the request method is GET (more on this later), the resource is located relative to the server root, and version 1.1 of HTTP protocol is used to communicate.

HTTP headers are special key-value pairs containing meta-data. Every header has a name and a value, which are separated by a colon. A header should occur only once. In the example above, the Host header helps the endpoint to choose the right server instance to serve the request. So the full, absolute resource locator was http://www.example.com/index.html. With the Accept header the client can signal which content types will it accept in the body of the response message. Basically, it describes the client's capabilities for the server. User-Agent is a common header to identify the client software. It can be used e.g. in statistics and to deliver different content specialized to different client software, for instance mobile devices.

Request body can attach additional information to the request. It can be form data, an uploaded file, or any other form of serialized data. Of course, the server can read this field of the message, and e.g. persist it to a database, or use it in another application-specific way. During GET requests it is usually empty, because this operation is safe and idempotent (more about this on the next slide). If the message body is present, an empty line must separate it from the headers.

After the request is completed the server starts the response. The end of the message is told to the client by simply closing the connection (in the simplest case).

Request methods

- 9 methods in total: HEAD, GET, POST, PUT, DELETE, TRACE, OPTIONS, CONNECT and PATCH
- The most important and most frequent methods:
  - GET: retrieve resources like HTML pages and associated content
  - POST: send form data or upload a file; the message body is filled with the corresponding data
- Safe methods: HEAD, GET, OPTIONS, TRACE
- Idempotent methods: safe methods + PUT and DELETE
- Methods used by REST: GET, PUT, POST, DELETE

The most commonly used method on the web is GET. It acquires a resource with a given URI, the resource itself will be returned in the response body. HEAD is the same as GET, except it does not return the message body, just the response headers. TRACE, OPTIONS, CONNECT and PATCH are used only in very special applications. PUT creates the specified resource with the given content, while DELETE removes it permanently.

POST is universal in the terms of processing: it can receive arbitrary data from the client and process any way the application needs it. For example, it can initiate a money transfer, log in a user or order a product. This makes repeating POST requests dangerous. For correct handling of POST requests see the POST-Redirect-GET section later. For historical reasons, in most of the cases POST is used to create and delete resources instead of their dedicated methods above.

Safe methods cannot change the state of the server. In the case of idempotent methods, repeating the same request multiple times will yield the same result as sending it only once. Of course, all safe methods are idempotent. As DELETE means removing a given resource, removing it twice will yield the same: the resource will not exist anymore. The case of PUT is the same: it creates a resource if it does not exists with the content from the request body. Creating the same thing twice at the same location with the same content does not change anything (the fact is e.g. the last modification date can change).

Although HTTP-conforming implementations must follow these rules regarding the request methods, the actual software can do anything, e.g. it is not guaranteed that a PUT request will create any resource in a given application.

Response

- Responses are sent from servers to clients
- Consists of three parts:
    1) Response line
    2) Response headers
    3) Optional message body after an empty separator line
- Very similar to a request

```
HTTP/1.1 200 OK
Content-Length: 38
Content-Type: text/html; charset=UTF-8

<html><body>Hello HTML!</body></html>
```

The structure of a response message resembles the request messages. First, it contains a response line, which contains the protocol version, response status code and its text representation. The status code indicates the success or failure of a request.

Headers have a similar role than in the case of a request: they contain meta-information, for example the length and content type of the response body. It can also contain the name and version of the server software, the last modification time of the given resource, and caching information.

Of course, the message body is used to send the requested content back to the client. The body itself is separated from the headers with an empty line, and its length is

indicated in the `Content-Length` header as the number of its raw octets (one octet is eight bits). This enables the clients to show e.g. a progress bar during the transfer. However, this header is not always present, e.g. in the case of streamed data. Message body is optional, for example in the case of a `HEAD` request it is not present.

Response status code

- Indicates the success or failure of a request
- Five categories based on the first number of the code
  - 1xx Informational
  - 2xx Successful
  - 3xx Redirection
  - 4xx Client error
  - 5xx Server error

Common status codes

- 101 Switching protocols
- 200 OK
- 201 Created
- 204 No Content
- 301 Moved Permanently
- 302 Found
- 303 See Other
- 304 Not Modified
- 307 Temporary Redirect
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 500 Internal Server Error

There are a lot of standard status messages which can be used. Status messages started by 1 are informational messages, like 101 which is a request to the client to change protocol. This is used in modern web applications to switch to WebSocket protocol.

Messages starting with 2 represent success. The standard message for a `GET` request is 200, for a `PUT` request is 201.

3xx messages are redirections: 301 means that this request and all future request to this `URL` should be made to an other given in the `Location` header. This is useful to inform search engines about a `URL` change. 302 means the resource can be found on an other `URL`. The specification said the browser must use the same method to access

the new URL, however the first web browsers implemented by switching to GET. This two behaviors are now reflected by 303 and 307. 303 means that the resource can be found on a new URL with GET method. 307 means that the resource can be found on a new URL with same method. 302 is still commonly used in web applications to create a POST-Redirect-GET pattern (see later).

Messages starting with 4 are client errors while 5xx messages are server-side errors.

Headers

Content negotiation

- The client can inform the server about its capabilities
- What kind of representations can be understood

Request

```
Accept: text/html,application/xhtml+xml;q=0.9,image/webp,*/*;q=0.8
Accept-Encoding: gzip,deflate
Accept-Language: hu,en-US;q=0.8,en;q=0.6
```

Response

```
Content-Type: text/html; charset=utf-8
Content-Encoding: gzip
```

With the headers of the Accept-* family, the client can tell the server which representations of the requested resource can it understand. The encoding, type and language can also be specified. Because each header can hold a set of multiple possibilities, the response also contains this information with the values actually chosen by the server. Encoding specifies how the data can be extracted from the response body. Its type describes how to interpret the data, e.g. as an image, web site, spreadsheet, sound, etc. Types are given as so-called MIME-types.

MIME types

- Originated from Multipurpose Internet Mail Extensions (MIME)
- Internet media type
- Shortly identifies the representation of a given data
- Specified in RFC2046
- General syntax:

```
top-level type name / subtype name [ ; parameters ]
```

Examples

```
image/png                   Portable Network Graphics image
image/svg+xml               Scalable Vector Graphics in XML format
text/html                   HTML document
text/plain; charset=utf-8   Plain text in UTF-8 encoding
```

MIME was used to extend the capabilities of the original e-mail messages, by making them able to contain different kind of contents in a single message. This way messages can contain attachments, embedded images, alternate contents like HTML and plain text representations in different character sets, etc. MIME types were created to identify the different kind of media types, by giving them a short, structured name. A standard MIME-type name consists of a top-level type name and a subtype name. Currently, the top-level name can be one of the following: application, audio, example, image, message, model, multipart, text and video. A complete list of the standard media types can be found at http://www.iana.org/assignments/media-types/media-types.xhtml, which is maintained by Internet Assigned Numbers Authority (IANA).

Cookies

- Primitive storage implemented in the client
- Binds a value to a name
- The server can instruct the client to manipulate it
- Data will be sent to the server with every request automatically
- Per-domain, limited storage time and size
- Useful for tracking clients

Last response

```
Set-Cookie: a=b; Expires=Mon, 04-May-2020 14:14:06 GMT; Path=/
Set-Cookie: SID=123983945; Path=/; domain=.example.com
```

Next request (same domain)

```
Cookie: a=b; SID=123983945
```

A cookie is a primitive storage which is able to bind a string value to a name in the client. The time and amount of cookies are very limited by the browsers, maximum cookie size is usually 4KB and the number of cookies is limited as well. Cookies without expiration date are called session cookies and are deleted when the browser window is closed.

Servers can instruct clients in response headers to create, modify or remove a given cookie. With each request message, the client will automatically send the contents of

the cookies corresponding to that given domain in request headers. This way server can use them to give an identifier token to each client. We will see this role of cookies later in session handling.

Caching

- Freshness
  - Fresh content is served from cache
  - Controlled by `Expires` and `Cache-Control` headers
  - Heuristics if there is `Last-Modified` header
- Validation
  - Checking freshness of a resource with the origin server
  - `Last-Modified` and `ETag` headers

`HTTP` supports caching resources in clients and intermediate servers (proxies) to reduce network latency and traffic. This mechanism is controlled by request and response headers. When the server sends an `Expires` header with an `HTTP` date, it explicitly specifies when the cache will need to re-validate its corresponding resource. `Cache-Control` header can use a `max-age` or `s-maxage` value to specify the freshness of a resource in seconds. In `HTTP 1.1`, this can override the freshness value set by `Expires`. If the `Last-Modified` header is present, heuristics can also be used to estimate expiry.

Checking whether a resource has changed since its last requested from the origin server is called validation. This can be done either using the date value of a `Last-Modified` header, or an entity tag sent in the `ETag` header. Entity tag is usually a hash generated from a state of the resource to identify it. When a resource changes, its entity tag also changes. If the entity tag changes only when the resource is changed in a way which makes it semantically different from its previous version, it is called a weak, otherwise a strong entity tag.

Cache validation

- Using conditional request
- Based on last modification time

Initial request

```
GET /resource HTTP/1.0
...
```

Initial response

```
HTTP/1.1 200 OK
Last-Modified: Wed, 13 Feb 2013 17:43:27 GMT
...
```

Cache validation

- Using conditional request
- Based on last modification time

Validation request

```
GET /resource HTTP/1.0
If-Modified-Since: Wed, 13 Feb 2013 17:43:27 GMT
...
```

Validation response

```
HTTP/1.1 304 Not Modified
...
```

In the initial response the server sends the last modification time of the resource. When the client wants to validate whether its version is fresh, it can issue a conditional request by sending back its last modification value in an `If-Modified-Since` header. If the resource is not changed since then, it will send back a response with status code 304 to indicate it is not modified, and the body of the message will be empty. Otherwise, the request will be handled as any other request.

Cache validation

- Using conditional request
- Based on entity tag

Initial request

```
GET /resource HTTP/1.0
...
```

Initial response

```
HTTP/1.1 200 OK
ETag: "237eb-2d7-4d59eadf71be4"
...
```

Cache validation

- Using conditional request
- Based on entity tag

Validation request

```
GET /resource HTTP/1.0
If-None-Match: "237eb-2d7-4d59eadf71be4"
...
```

Validation response

```
HTTP/1.1 304 Not Modified
...
```

This method uses an entity tag to validate the resource instead of its last modification time. Initial response will contain the entity tag corresponding to the requested resource as the value of an `ETag` header. When validation occurs it can be used to issue a conditional request using an `If-None-Match` header. If the current entity tag of the resource on the origin server matches one of those sent in `If-None-Match`, the server will respond with an 304 status code, which means the given resource is still fresh.

Important mechanisms

`POST`-Redirect-`GET` (`PRG`)

- Problem:
  1) Client sends form data as a `POST` request
  2) Then refreshes the page which was returned in the response
  3) Browser will send the `POST` request again with the same data
  - In most of the cases it is unwanted an dangerous
  - E.g. a money transfer can be requested twice
- Solution:
  - Never send content as a response to a `POST` request
  1) Use redirection, send status code 302 or 303 in response to a `POST`
  2) The client will fetch the new target `URI` it with `GET`
  3) The resulting page can be refreshed safely
- Design pattern

For example, to create a web application which transfers money, it is reasonable to implement the request for money transfer with a `POST` request. However, this request will appear in the browsing history and by pushing the back button on the browser, the transaction can be repeated incidentally. The `POST`-Redirect-`GET` pattern is a common way to solve this problem, by redirecting the client after the `POST` request to an other page, which is accessed with `GET`. This page shows probably the result of the transaction, and can be safely refreshed. When the user presses the back button, the form will be showed again, and it will not be posted automatically.

Redirection is done by sending a 3xx response code and a `Location` header with a target `URI`. Status code 303 (See Other) explicitly means that client must request the target `URI` with `GET` method. Using 302 (Found) is incorrect in terms of the `HTTP 1.1` standard, but for historical reasons it is often used by many web applications and browsers will handle it the same way as 303.

Session handling

- Separated storage for each client on the server
  - Store data between multiple requests and responses
  - Carry authorization information
  - May be authenticated using e.g. a log-in form
- Storing data
  - In-memory
  - Regular file
  - Database
- A session identifier is assigned to each client in a cookie
  - The client sends it automatically with every request
  - The server can access the appropriate storage by the identifier
  - Cookie removed when the browser closed

Sessions are widely used in web applications to store intermediate data on the server separately for each client. The server connects a client with its storage by a generated session identifier. By storing it in a cookie the server ensures every subsequent request will contain it until the end of the session. A session can usually end in three ways. Because the cookie storing the identifier is usually set with expiry zero, it will be deleted when the browser is closed. These cookies are called session cookies. The server can also directly remove the cookie, for example by responding to a log-out request. Sessions usually have a timeout configured on the server, which is the maximum time amount that can be elapsed between two subsequent requests in the session. When a client issues a request with a session identifier that is timed out, the server will also remove its session cookie. Whether the server assigns a new session automatically or not, it depends on the actual application.

Building web applications

CGI

Introduction

- Common Gateway Interface (CGI)
- Some of the HTTP requests are processed by an external program
- Web server forwards the request and response as a relay
  - Request headers are passed as environment variables
  - Request body as standard input
  - Response headers and body are written to the standard output
- One process per request
  - Cost of process management is high, makes it slow
  - FastCGI serves multiple requests with the same process
- Useful for:
  - Generating dynamic content
  - Processing form data
  - Managing databases, etc.

Common Gateway Interface enables the web server to outsource the processing of certain HTTP requests to external software. Usually there is a special directory (often called cgi-bin) in the server configuration, that can be used to look up an external program which should be used to serve requests for a given URI. These programs are usually written in interpreted scripting languages like Perl, but using binaries is also possible.

When the web server receives a request, it analyzes the request URI. If it points inside the cgi-bin directory, it starts the given program as a new process, passing the request headers as environment variables, and request body on the standard input stream. After the program exited successfully, its standard output will be read by the web server and forwarded to the client. If the program in question is not found, response status will be 404 (Not Found). When an error occurs during the processing, an 500 Internal Server Error will be reported to the client.

Unlike vendor-specific web server modules or plugins, CGI makes web applications portable between different server software. Its biggest drawback is its one request - one process mapping. FastCGI eliminated this problem by using a pool with a limited number of processes to serve multiple requests.

Example

```
#!/bin/bash

cat << EOF
Content-type: text/html

<html>
<head>
<title>Hello World</title>
</head>
<body>
Hello World!
</body>
</html>
EOF
```

This example shows a simple CGI script written in Bash shell scripting language. It simply outputs HTML content with a single Content-type header. The web server will extend it with additional headers and a response line, and sends it back to client.

Servlets

Introduction

- Servlet
  - An object that extends a server's processing capabilities
  - Not only for HTTP, but this is the most common use case
  - Higher integration than with CGI, different processing model
- Part of Java EE
- Runs in a Servlet Container
  - Manages configuration and lifecycle
  - Glassfish, Tomcat, Jetty, etc.
- With other important technoloigies
  - JavaServer Pages (JSP)
  - JSP Standard Tag Library (JSTL)
  - JavaServer Faces (JSF)

Servlet specification

- Versions
  - Servlet 2.5 (Java EE 5): annotation support
  - Servlet 3.0 (Java EE 6): asynchronous servlets, file uploads
  - Servlet 3.1 (Java EE 7): non-blocking I/O
- General API in `javax.servlet.*` package
- HTTP-related interfaces in `javax.servlet.http.*`

HTTP Servlet API

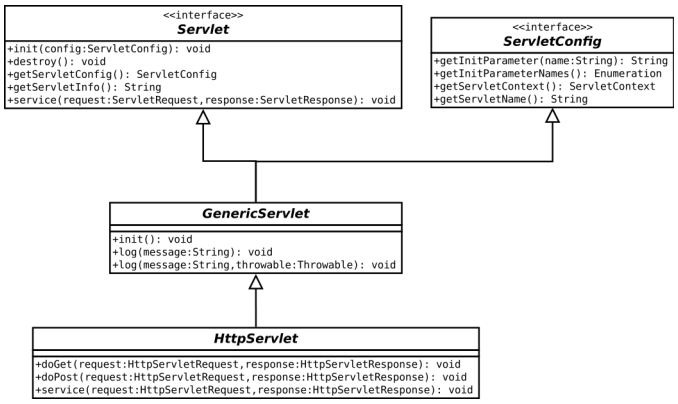


Figure 2: Structure of the HTTP Servlet API

Every servlet implements the `javax.servlet.Servlet` interface. This provides a `service` method, which is parametrized with a request and response object. Initially, the servlet container instantiates the actual servlet class for every URL mapping (see later), and calls the `init` method with a configuration object. These configurations can be used to pass parameters to servlets at initialization time. On every request, the container prepares a request and a response object, and calls the `service` method of the appropriate servlet object.

Because servlets are used mostly in web applications, a special class named `javax.servlet.http.HttpServlet` can be derived to implement custom application logic. It provides a dispatcher code in the `service` method which automatically calls its `doXXX` method, where `XXX` is the name of the current HTTP request method. For example, on a GET request the `service` method will call `doGet`, on POST it will call `doPost`. These methods are parametrized with HTTP request and response classes, which makes it easy to manage various parts of messages, including headers and body.

Example

```
import javax.servlet.*; import javax.servlet.http.*;
import java.io.*;
public class Hello extends HttpServlet {
  private String name;
  public void init() {
    name = (String)getServletConfig().getInitParameter("name");
  }
  public void doGet(HttpServletRequest req, HttpServletResponse res)
      throws IOException {
    res.setContentType("text/html");
    PrintWriter out = res.getWriter();
    out.println("<html><head><title>Hi</title>" +
                "</head><body>Hello " + hello + "</body></html>");
    out.close();
  }
}
```

This simple servlet responds only to requests sent with `GET` `HTTP` method. It returns a simple and short `HTML` content, by writing it into the output stream of the response object. It also sets the value of `Content-Type` header. In case of other methods, it responds with status code 405 (Method Not Allowed).

Every servlet can have initialization parameters. These parameters can be configured by the web deployment descriptor (`web.xml`). The descriptor is also responsible to map the servlet class to `URL` patterns.

Configuration in `web.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://java.sun.com/xml/ns/javaee" ...>
    <servlet>
        <servlet-name>hello-servlet</servlet-name>
        <servlet-class>Hello</servlet-class>
        <init-param>
            <param-name>name</param-name>
            <param-value>World</param-value>
        </init-param>
    </servlet>
    <servlet-mapping>
        <servlet-name>hello-servlet</servlet-name>
        <url-pattern>/sayhello</url-pattern>
    </servlet-mapping>
</web-app>
```

The servlet with a specific configuration is named. In this case it is called `hello-servlet`. Such a configuration can be mapped to one or more `URL` pattern, which is relative to the root of the application. The root of the application is usually not the server root, but a sub-path of it, and it is called the context path. The context path is usually given during deployment, however it is possible to configure an application to more than one context path.

URL-mapping

- Can be specified in
  - Configuration file (`web.xml`)
  - Annotation on servlet class (`@WebServlet`)

```
@WebServlet(urlPatterns={"/sayhello"})
public class Hello extends HttpServlet ...
```

- Mapping possibilities
  - Fixed `URL`: `/hello`
  - By sub-path: `/path/to/servlet/*`
  - By extension: `*.gif`
  - Default servlet: `/`

There are four kinds of `URL` mapping. The first one is a fixed, exact `URL`, for example `/hello`. The other one ends with an asterisk and maps a servlet to a subpath of the `URL`. The third one is which handles a specific file extension, such as `gif`. The last one is called the default servlet. The default servlet is called when there was no mapping defined for the requested `URL`.

Deployment

- Web applications are packaged into archives
  - Web Application Archive (`WAR`)
  - An ordinary `JAR` file with special content
  - Can be created with the `jar` command
- File hierarchy:

```
index.html
logo.png
META-INF/
    MANIFEST.MF
```

```
WEB-INF/
    web.xml
    classes/
        Hello.class
    lib/
        any-external-library.jar
```

To deploy the web application, we have to create a `war` file, which is created by the `jar` utility. The `war` file, however, has a different structure. The files are directly accessible from the web, except the directories `WEB-INF` and `META-INF`. If not stated otherwise, the context path of the application will be the name of the `war` file.

`EJB`s can be injected the servlets as well, if deployed in the same Enterprise Application Archive (`EAR`) module. The `war` can define their own `EJB`s as well, however, some features may be missing (like the possibility of invocation from outside the same virtual machine). This restricted mode is called `EJB` lite.

Request object

- Implements `javax.servlet.http.HttpServletRequest`
- Contains information about the current request
- Provides getters to extract information about
  - Request line: `getMethod()`, `getRequestURI()`, `getQueryString()`
  - Headers: `getHeaderNames()`, `getHeaders(name)`
  - Cookies: `getCookies()`
  - Body contents: `getInputStream()`
  - Parameters: `getParameterNames()`, `getParameter(name)`

Response object

- Implements `javax.servlet.http.HttpServletResponse`
- Builds up the response
- Provides setters to adjust its contents
  - Response line: `setStatus(code)`
  - Headers: `setHeader(name, value)`
  - Cookies: `addCookie(cookie)`
  - Body contents: `getOutputStream()`
- Contains all `HTTP` status codes as constants prefixed with `SC_`
- Provides `URL` encoding facilities: `encodeURL(url)`

Redirection

- Send the standard precofigured error page:
  - `response.sendError(code)`
  - Code is from `HttpServletResponse.SC_*`, from 4xx or 5xx category
- External redirection
  - `response.sendRedirect(uri)`
  - Sends a `Location` header with the new `URI`
  - Client will fetch the new `URI`
- Internal redirection
  - `request.getRequestDispatcher(path).forward(request, response)`
  - Redirects inside the server, the client cannot see it
  - To other servlet, `JSP`, `HTML` file…

Scopes

- Stores for `JavaBeans` with different lifetime and accessibility
- Mapping a name to an object
- Communication between servlets and `JSP`s
- Different types:
  - Application scope
  - Session scope
  - Request scope
  - Page scope (used in `JSP`)
  - Flash scope (theoretical)

Servlet containers provide a very useful storage mechanism for storing `JavaBeans`. These stores are called scopes. Some scopes, like application scope can hold objects for a very long time, and we can access those objects from various parts of the application. Others, like request scope just store their contents for the time of a single request, and these objects can be accessed only from where the request object itself is accessible. The lifetime and visibility of objects in a given scope is in descending order on the slide, except the theoretical `flash` scope.

Application scope

- Available while the application is running
- Objects accessible everywhere where the `ServletContext` is
- Content is shared among all clients

```
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
    ...
    MyBean b1 = new MyBean(...);
    getServletContext().setAttribute("name", b1);
    ...
    MyBean b2 = (MyBean)getServletContext().getAttribute("name");
    ...
}
```

Application scope is accessible through an instance of `ServletContext`. This object enables a servlet to communicate with its container. Methods `getAttribute` and `setAttribute` are the same on all scopes. By default, lifetime of objects in the application scope equals with the lifetime of the whole application - except when an object is explicitly removed with a call to method `removeAttribute(name)`. It can also be found on all scopes.

Session scope

- Available in the current browser session
  - Implemented by Session Cookies
  - Ends when time out
  - Stored objects must be `java.io.Serializable`

```
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
    ...
    MyBean b1 = new MyBean(...);
    req.getSession().setAttribute("name", b1);
    ...
    MyBean b2 = (MyBean)req.getSession().getAttribute("name");
    ...
}
```

The server uses a session cookie to identify the browser session (but the data is stored on the server side). There is no explicit method to detect if a remote browser is closed, so it is freed, when a timeout occurs. It is possible that the server serializes session data to save memory, so data stored in the session should implement `java.io.Serializable`. The `Session` object, which can be retrieved by `req.getSession()` provides other useful methods to e.g. renew the session identifier, etc.

Request scope

- Available in the current HTTP request
  - Transfer data between internal redirects
  - Between filters, servlets and JSPs

```
public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
    ...
    MyBean b1 = new MyBean(...);
    req.setAttribute("name", b1);
    ...
    MyBean b2 = (MyBean)req.getAttribute("name");
    ...
}
```

JavaServer Pages

- Generating long HTML output in a servlet is not recommended
  - The whole output is in string literals
  - Needs a lot of string concatenation
  - Makes maintenance of servlet code hard
- Solution: separation of business logic and presentation
  - Write templates where default mode is to output contents
  - Variables and code can be inserted with special marks
  - They can be compiled into servlets automatically
  - Servlets can prepare data and forward to JSP pages internally

Example

```
<html>
<body>
<%
// Inserted as ordinary Java code inside a servlet
double x = Math.sqrt(2);
if ('GET'.equals(request.getMethod())) {
%>
    Hello ${name}, the request method is GET!
<%
}
%>
```

```
    Current page: ${request.getRequestURI()}
    Square root of 2 is ${x}.
</body>
</html>
```

The <% and %> marks the start and end of a scriptlet: it is Java code, which will be inserted into the method body of a servlet. Expression language can also be used like `${variable}`. Variables will be looked up in all of the scopes in the following order: page, request, session, application. If the variable is not found in either scope, expression will be substituted with a `null` value.

Page scope

- Available in the current JSP page

```
<html>
<body>
<%
// available in the current page
MyBean b = new MyBean();
%>
    Test: ${b.someField.someOtherNestedField}
</body>
</html>
```

The declaration inside a scriptlet will compiled to a local variable of a servlet method. Its lifetime and visibility is limited to the current page only.

Implementing POST-Redirect-GET

```
@Override
protected void doGet(HttpServletRequest req,
                     HttpServletResponse resp)
    throws ServletException, IOException {
  // prepare
  req.setAttribute("bean", "result");
  getServletContext().getRequestDispatcher("/WEB-INF/jsp/form.jsp")
    .forward(req, resp);
}
@Override
protected void doPost(HttpServletRequest req,
                      HttpServletResponse resp)
    throws ServletException, IOException {
```

```
  // process
  resp.sendRedirect(req.getContextPath()+req.getServletPath());
}
```

Implementing POST-Redirect-GET

```
<html>
<body>
  <form action="" method="post">
      <input type="text" name="name" value="${bean.value}"/>
      <input type="submit" name="submit" value="Save"/>
  </form>
</body>
</html>
```

This example is an implemented POST-Redirect-GET. The JSP file is put to the WEB-INF folder, so it is protected from direct access. The servlet has full control of which JSP file can display the data, and collects the data and pushes it to the JSP file. This is called PUSH model. An other alternative is JSF, where the JSF file is directly accessible and it pulls the data from the backend. The PUSH model has more control over the flow, however the PULL model work much better with components. Do not confuse these models with HTTP's pull and push!

The doPost method processes the form submission. After a success, the browser is redirected to the original form using GET method.

Flash pseudo-scope

- After a POST-Redirect-GET, on the result page sometimes we need to show some data, which is generated during processing of POST
- We need to use the session scope to avoid sharing data between clients but save it between processing and the GET after redirection
- After a single use, we can clear the data
- The data just ``flashes'' in the scope for a single subsequent request
- It can be simulated with session scope and filters

JavaServer Faces

- Similar technique to JSP, but more advanced
- Describes UI component models in XML files
- View templates
- Automatic view state management
- Uses events and backing beans